

웹시스템설계및개발1

this



상황에 따라 달라지는 this



● 객체지향 언어의 this

- 클래스(class)로 생성한 인스턴스(instance)를 의미
- 클래스에서만 사용할 수 있음

● 자바스크립트의 this

- 어디서든 사용할 수 있음
- 상황에 따라 this가 바라보는 대상이 달라짐

● this는 실행 컨텍스트가 생성될 때 함께 결정

브라우저 환경에서 전역 공간의 this



● 전역 객체

- 전역 컨텍스트를 생성하는 주체
- 자바스크립트 런타임 환경에 따라 다른 이름과 정보를 가지고 있음

● 브라우저 환경에서의 전역 객체

- window
- 전역 공간에서의 this는 window를 바라봄

```
Elements Console Sources Network Performance Memory Application
top Filter Default levels ▼
> console.log(this)
  ▶ Window {parent: Window, opener: null, top: Window, length: 4, frames: Window, ...}
< undefined
> console.log(window)
  ▶ Window {parent: Window, opener: null, top: Window, length: 4, frames: Window, ...}
< undefined
> console.log(this === window)
  true
< undefined
```

Node.js 환경에서 전역 공간의 this



JS test.js

C: > Users > wkimc > Documents > JS test.js

```
1 console.log(this);
2 console.log(global);
3 console.log(this === global);
```

OUTPUT TERMINAL DEBUG CONSOLE PROBLEMS

C:\Program Files\nodejs\node.exe test.js

{}

```
Object [global] {
  global: [Circular],
  clearInterval: [Function: clearInterval],
  clearTimeout: [Function: clearTimeout],
  setInterval: [Function: setInterval],
  setTimeout: [Function: setTimeout] { [Symbol(util.promisify.custom)]: [Function] },
  queueMicrotask: [Function: queueMicrotask],
  clearImmediate: [Function: clearImmediate],
  setImmediate: [Function: setImmediate] {
    [Symbol(util.promisify.custom)]: [Function]
  }
}
false
```

전역 공간에서의 this



- 브라우저 환경에서 전역 변수를 선언하면 자바스크립트 엔진은 이를 전역 객체의 속성으로 할당

```
Elements Console
top
> var a = 1;
< undefined
> console.log(a);
1
< undefined
> console.log(window.a);
1
< undefined
> console.log(this.a);
1
```

```
1 var a = 1;
2 console.log(a);
3 console.log(global.a);
4 console.log(this.a);
```

```
OUTPUT TERMINAL DEBUG CONSOLE PROBLEMS
C:\Program Files\nodejs\node.exe test.js
1
undefined
undefined
```


전역 공간에서의 this



● 자바스크립트의 모든 변수는 특정 객체의 속성으로서 동작

- 실행 컨텍스트는 변수를 수집해서 Lexical Environment의 속성으로 저장
- 변수를 호출하면 Lexical Environment를 조회해서 일치하는 속성이 있을 경우 그 값을 반환
- 브라우저 환경에서의 전역 컨텍스트의 경우 Lexical Environment는 전역 객체를 참조

● var로 변수를 선언하는 대신 window의 속성에 직접 할당하면?

```
> var a = 1;
< undefined
> window.b = 2;
< 2
> console.log(a, window.a, this.a);
1 1 1
< undefined
> console.log(b, window.b, this.b);
2 2 2
< undefined
```

전역 공간에서의 this



- 전역 변수 선언과 전역 객체의 속성 할당 사이에 전혀 다른 경우
 - '삭제' 명령
 - var로 선언한 전역 변수와 전역 객체의 속성은 호이스팅 여부 및 삭제 가능 여부에서 차이를 가짐

```
> var a = 1;
< undefined
> delete window.a;
< false
> console.log(a, window.a, this.a);
1 1 1
< undefined
```

```
> window.c = 3;
< 3
> delete window.c;
< true
> console.log(c, window.c, this.c);
✖ ▶ Uncaught ReferenceError: c is not defined
   at <anonymous>:1:13
```

```
> var b = 2;
< undefined
> delete b;
< false
> console.log(b, window.b, this.b);
2 2 2
< undefined
```

```
> window.d = 4;
< 4
> delete d;
< true
> console.log(d, window.d, this.d);
✖ ▶ Uncaught ReferenceError: d is not defined
   at <anonymous>:1:13
```

메소드로서 호출할 때 메소드 내부에서의 this



함수

- 그 자체로 독립적인 기능을 수행

메소드

- 자신을 호출한 대상 객체에 관한 동작을 수행

```
1
2  var func = function () {
3    |   console.log(this === global);
4  |   }
5
6  func();
7
```

OUTPUT TERMINAL DEBUG CONSOLE PROBLEMS

C:\Program Files\nodejs\node.exe test.js
true

```
1
2  var func = function () {
3    |   console.log(this === global);
4    |   console.log(this === obj)
5  |   }
6
7  var obj = {
8    |   method: func
9  |   };
10
11  obj.method();
12
```

OUTPUT TERMINAL DEBUG CONSOLE PROBLEMS

C:\Program Files\nodejs\node.exe test.js
false
true

메소드로서 호출할 때 메소드 내부에서의 this



- 어떤 함수를 호출할 때 그 함수 이름 (속성명) 앞에 객체가 명시되어 있는 경우에는 메소드로 호출한 것이고, 그렇지 않은 모든 경우에는 함수로 호출한 것

```
1
2  var obj = {
3      method: function() {
4          console.log(this)
5      }
6  };
7
8  obj.method();
9  obj['method']()
10
```

OUTPUT TERMINAL DEBUG CONSOLE PROBLEMS

```
C:\Program Files\nodejs\node.exe test.js
{ method: [Function: method] }
{ method: [Function: method] }
```

메소드로서 호출할 때 메소드 내부에서의 this



- this에는 호출한 주체에 대한 정보가 담김
- 어떤 함수를 메소드로서 호출하는 경우 호출 주체는 객체

```
1
2  ✓ var obj = {
3      methodA: function() { console.log(this); },
4      inner: {
5          methodB: function() { console.log(this); }
6      }
7  };
8
9  obj.methodA();
10 obj['methodA']()
11
12 obj.inner.methodB();
13 obj.inner['methodB']();
14 obj['inner'].methodB();
15 obj['inner']['methodB']();
16
```

```
{
  methodA: [Function: methodA],
  inner: { methodB: [Function: methodB] }
}
{
  methodA: [Function: methodA],
  inner: { methodB: [Function: methodB] }
}
{ methodB: [Function: methodB] }
{ methodB: [Function: methodB] }
{ methodB: [Function: methodB] }
{ methodB: [Function: methodB] }
```

함수로서 호출할 때 함수 내부에서의 this



- 어떤 함수를 함수로서 호출할 경우에는 this가 지정되지 않음
 - 함수로서 호출하는 것은 호출 주체 (객체지향 언어에서의 객체)를 명시하지 않고 개발자가 코드에 직접 관여해서 실행한 것이기 때문에 호출 주체를 알 수 없음
 - 실행 컨텍스트를 활성화할 당시 this가 지정되지 않은 경우 this는 전역 객체를 바라보기에 함수에서의 this는 전역 객체를 가리킴

함수로서 호출할 때 함수 내부에서의 this



● 메소드의 내부 함수에서의 this

```
1
2  var obj1 = {
3
4      outer: function () {
5
6          console.dir(this, { depth: 0 });
7
8          var innerFunc = function () {
9              console.dir(this, { depth: 0 });
10             };
11
12             innerFunc();
13
14             var obj2 = {
15                 innerMethod: innerFunc,
16             };
17
18             obj2.innerMethod();
19         }
20     };
21
22     obj1.outer();
23
```

```
{ outer: [Function: outer] }
Object [global] {
  global: [Circular],
  clearInterval: [Function: clearInterval],
  clearTimeout: [Function: clearTimeout],
  setInterval: [Function: setInterval],
  setTimeout: [Function],
  queueMicrotask: [Function: queueMicrotask],
  clearImmediate: [Function: clearImmediate],
  setImmediate: [Function]
}
{ innerMethod: [Function: innerFunc] }
```

함수로서 호출할 때 함수 내부에서의 this



● 메소드의 내부 함수에서의 this를 우회하는 방법

```
1
2  var obj = {
3
4      outer: function () {
5
6          console.dir(this, { depth: 0 });
7
8          var innerFunc1 = function () {
9              console.dir(this, { depth: 0 });
10             };
11
12             innerFunc1();
13
14             var self = this;
15
16             var innerFunc2 = function () {
17                 console.dir(self, { depth: 0 });
18             };
19
20             innerFunc2();
21         },
22     };
23
24     obj.outer();
25
```

```
{ outer: [Function: outer] }
Object [global] {
  global: [Circular],
  clearInterval: [Function: clearInterval],
  clearTimeout: [Function: clearTimeout],
  setInterval: [Function: setInterval],
  setTimeout: [Function],
  queueMicrotask: [Function: queueMicrotask],
  clearImmediate: [Function: clearImmediate],
  setImmediate: [Function]
}
{ outer: [Function: outer] }
```


함수로서 호출할 때 함수 내부에서의 this



- 호출 주체가 없을 때는 자동으로 전역 객체를 바인딩하지 않고 호출 당시 주변 환경의 this를 그대로 상속받아 사용할 수 있는 방법은?
 - 현재 컨텍스트의 this에 바인딩된 대상이 없으면 직전 컨텍스트의 this를 바라보도록 하는 방법은?
- this를 바인딩하지 않는 함수
 - 화살표 함수 (arrow function)는 실행 컨텍스트를 생성할 때 this 바인딩 과정 자체가 빠지게 되어, 상위 스코프의 this를 그대로 활용

함수로서 호출할 때 함수 내부에서의 this



● this를 바인딩하지 않는 함수

```
1
2  var obj = {
3
4      outer: function () {
5
6          console.log(this);
7
8          var innerFunc = () => {
9              console.log(this);
10         };
11
12         innerFunc();
13     }
14 };
15
16 obj.outer();
17
```

```
{ outer: [Function: outer] }
{ outer: [Function: outer] }
```

생성자 함수 내부에서의 this



- new 명령어와 함께 함수를 호출하면 해당 함수가 생성자로 동작
- 어떤 함수가 생성자 함수로서 호출된 경우 내부에서의 this 는 새로 만들 구체적인 인스턴스 자신이 됨

```
1
2 var Cat = function (name, age) {
3   this.bark = '야옹';
4   this.name = name;
5   this.age = age;
6 };
7
8 var choco = new Cat('초코', 7);
9 var nabi = new Cat('나비', 5);
10 console.log(choco, nabi);
11
```

```
1
2 class Cat {
3   constructor(name, age) {
4     this.bark = '야옹';
5     this.name = name;
6     this.age = age;
7   }
8 }
9
10 var choco = new Cat('초코', 7);
11 var nabi = new Cat('나비', 5);
12 console.log(choco, nabi);
13
```

Cat { bark: '야옹', name: '초코', age: 7 } Cat { bark: '야옹', name: '나비', age: 5 }

명시적으로 this를 바인딩하는 방법



● call 메소드

- call 메소드의 첫 번째 인자를 this로 바인딩하고, 이 후의 인자들을 호출할 함수의 매개변수로 전달

`Function.call()`

```
(method) Function.call(this: Function, thisArg: any, ...argArray: any[]): any
```

Calls a method of an object, substituting another object for the current object.

@param `thisArg` — The object to be used as the current object.

@param `argArray` — A list of arguments to be passed to the method.

명시적으로 this를 바인딩하는 방법



● call 메소드

```
1
2  var func = function (a, b, c) {
3      console.dir(this, { depth:-1 });
4      console.log(a, b, c)
5  };
6
7  func(1, 2, 3);
8
```

OUTPUT TERMINAL DEBUG CONSOLE PROBLEMS

```
C:\Program Files\nodejs\node.exe test.js
[Object [global]]
1 2 3
```

```
1
2  var func = function (a, b, c) {
3      console.dir(this, { depth: 0 });
4      console.log(a, b, c)
5  };
6
7  func.call({ o: 'newObj' }, 1, 2, 3);
8
```

OUTPUT TERMINAL DEBUG CONSOLE PROBLEMS

```
C:\Program Files\nodejs\node.exe test.js
{ o: 'newObj' }
1 2 3
```


명시적으로 this를 바인딩하는 방법



● call 메소드

```
1
2  var obj = {
3      a: 1,
4      method: function (x, y) {
5          console.log(this.a, x, y);
6      }
7  };
8
9
10 obj.method(2, 3);
11 obj.method.call({ a: 5 }, 2, 3);
12
```

OUTPUT

TERMINAL

DEBUG CONSOLE

PROBLEMS

C:\Program Files\nodejs\node.exe test.js

1 2 3

5 2 3

명시적으로 this를 바인딩하는 방법



● apply 메소드

- call 메소드와 기능적으로 동일
- 두 번째 인자를 배열로 받아 그 배열의 요소들을 호출할 함수의 매개 변수로 지정

```
1
2 var func = function (a, b, c) {
3   console.log(this);
4   console.log(this.x, a, b, c)
5 };
6
7 func.apply({ x: 2 }, [3, 4, 5]);
8
```

OUTPUT TERMINAL DEBUG CONSOLE PROBLEMS

```
C:\Program Files\nodejs\node.exe test.js
{ x: 2 }
2 3 4 5
```

```
1
2  var obj = {
3
4     x: 1,
5
6   method: function (a, b, c) {
7     console.log(this);
8     console.log(this.x, a, b, c);
9   }
10 };
11
12 obj.method.apply({ x: 2 }, [3, 4, 5]);
13
```

OUTPUT TERMINAL DEBUG CONSOLE PROBLEMS

```
C:\Program Files\nodejs\node.exe test.js
{ x: 2 }
2 3 4 5
```

call / apply 메소드의 활용



● 유사배열객체 array-like object에 배열 메소드를 적용

- 객체에는 배열 메소드를 적용할 수 없음
- 그러나 키가 0 또는 양의 정수인 속성이 존재하고 length 속성 값이 0또는 양의 정수인 객체, 즉 배열의 구조와 유사한 객체의 경우 (유사배열객체) call, apply 메소드를 이용해 배열 메소드를 사용할 수 있음

call / apply 메소드의 활용



```
1
2  var obj = {
3      0: 'a',
4      1: 'b',
5      2: 'c',
6      length: 3
7  };
8
9  Array.prototype.push.call(obj, 'd');
10 console.log(obj);
11
12 var arr1 = Array.prototype.slice.call(obj, 1, 3);
13 console.log(arr1);
14
15 var arr2 = Array.prototype.slice.call(obj);
16 console.log(arr2);
17
```

OUTPUT TERMINAL DEBUG CONSOLE PROBLEMS

```
C:\Program Files\nodejs\node.exe test.js
{ '0': 'a', '1': 'b', '2': 'c', '3': 'd', length: 4 }
[ 'b', 'c' ]
[ 'a', 'b', 'c', 'd' ]
```

call / apply 메소드의 활용



- 함수 내부에서 접근할 수 있는 arguments 객체도 유사배열 객체

```
1
2  function a() {
3      console.log(arguments)
4
5      var argv = Array.prototype.slice.call(arguments);
6      argv.forEach(function (arg) {
7          console.log(arg);
8      });
9  }
10
```

OUTPUT TERMINAL DEBUG CONSOLE PROBLEMS

C:\Program Files\nodejs\node.exe test.js

[Arguments] { '0': 1, '1': 2, '2': 3 }

1

2

3

call / apply 메소드의 활용



- 문자열도 배열 처럼 인덱스와 length 속성을 가지고 있으므로 call / apply 메소드를 활용할 수 있음
- 문자열의 경우 length 속성이 읽기 전용이므로 원본 문자열에 변경을 가하는 메소드(push, pop 등)는 에러를 던짐

```
1
2  var str = 'abc def';
3  Array.prototype.push.call(str, ', pushed string');
4
```

OUTPUT TERMINAL DEBUG CONSOLE PROBLEMS

C:\Program Files\nodejs\node.exe test.js

c:\Users\wkimc\Documents\test.js:3

Array.prototype.push.call(str, ', pushed string');

^

TypeError: Cannot assign to read only property 'length' of object '[object String]'

call / apply 메소드의 활용



```
1
2  var str = 'abc def';
3
4  var v= Array.prototype.every.call(str, function (char) {
5    |   return char !== ' ';
6  });
7
8  console.log(v);
9
10 var v = Array.prototype.some.call(str, function (char) {
11   |   return char !== ' ';
12 });
13
```

OUTPUT TERMINAL DEBUG CONSOLE PROBLEMS

```
C:\Program Files\nodejs\node.exe test.js
false
true
```

call / apply 메소드의 활용

```
1
2   var str = 'abc def';
3
4   var newArr = Array.prototype.map.call(str, function (char) {
5     |   return char + '!';
6   });
7
8   console.log(newArr);
9
10  var newStr = Array.prototype.reduce.call(
11    |   str,
12    |   function (string, char, i) {
13    |     |   return string + char + i;
14    |     },
15    |     '',
16  );
17
18  console.log(newStr);
19
```

OUTPUT TERMINAL DEBUG CONSOLE PROBLEMS

C:\Program Files\nodejs\node.exe test.js

```
[
  'a!', 'b!',
  'c!', '!',
  'd!', 'e!',
  'f!'
]
```

a0b1c2 3d4e5f6

call / apply 메소드의 활용



- ES6에서는 유사배열객체 또는 순회 가능한 모든 종류의 데이터 타입을 배열로 전환하는 Array.from 메소드를 도입

```
1
2   var obj = {
3       0: 'a',
4       1: 'b',
5       2: 'c',
6       length: 3,
7   };
8
9   var arr = Array.from(obj);
10  console.log(arr);
11
```

OUTPUT TERMINAL DEBUG CONSOLE PROBLEMS

```
C:\Program Files\nodejs\node.exe test.js
[ 'a', 'b', 'c' ]
```

call / apply 메소드의 활용



- 생성자 내부에서 다른 생성자를 호출

```
1
2 function Person(name, gender) {
3     this.name = name;
4     this.gender = gender;
5 }
6
7 function Student(name, gender, school) {
8     Person.call(this, name, gender);
9     this.school = school;
10 }
11
12 function Employee(name, gender, company) {
13     Person.apply(this, [name, gender]);
14     this.company = company;
15 }
16
17 var wkim = new Student('원빈', 'male', '건국대');
18 var jlim = new Employee('지영', 'female', '이화여대');
19
20 console.log(wkim);
21 console.log(jlim);
22
```

OUTPUT TERMINAL DEBUG CONSOLE PROBLEMS

C:\Program Files\nodejs\node.exe test.js

Student { name: '원빈', gender: 'male', school: '건국대' }

Employee { name: '지영', gender: 'female', company: '이화여대' }

call / apply 메소드의 활용



- 여러 인수를 묶어 하나의 배열로 전달하고 싶을 때 apply 활용

```
1
2  var numbers = [10, 20, 3, 16, 45];
3  var max = (min = numbers[0]);
4
5  numbers.forEach(function (number) {
6      if (number > max) {
7          max = number;
8      }
9      if (number < min) {
10         min = number;
11     }
12 });
13
14 console.log(max, min);
15
```

OUTPUT TERMINAL DEBUG CONSOLE PROBLEMS

C:\Program Files\nodejs\node.exe test.js
45 3

```
1
2  var numbers = [10, 20, 3, 16, 45];
3  var max = Math.max.apply(null, numbers);
4  var min = Math.min.apply(null, numbers);
5  console.log(max, min);
6
```

OUTPUT TERMINAL DEBUG CONSOLE PROBLEMS

C:\Program Files\nodejs\node.exe test.js
45 3

```
1
2  var numbers = [10, 20, 3, 16, 45];
3  var max = Math.max(...numbers);
4  var min = Math.min(...numbers);
5  console.log(max, min);
6
```

OUTPUT TERMINAL DEBUG CONSOLE PROBLEMS

C:\Program Files\nodejs\node.exe test.js
45 3

bind 메소드



- call 메소드와 비슷하지만 즉시 호출하지는 않고 넘겨 받은 this 및 인수들을 바탕으로 새로운 함수를 반환하기만 하는 메소드
- 새로운 함수를 호출할 때 인자를 넘기면 그 인수들은 bind 메소드를 호출할 때 전달했던 인수들의 뒤에 이어서 등록됨
- 즉, bind 메소드는 함수에 this를 미리 적용하는 것과 부분 적용함수를 구현하는 두 가지 목적을 지님

bind 메소드



```
1
2  var func = function (a, b, c, d) {
3      |   console.log(this, a, b, c, d);
4  };
5
6  var bindFunc1 = func.bind({ x: 'bindFunc1' });
7  bindFunc1(1, 2, 3, 4);
8
9  var bindFunc2 = func.bind({ x: 'bindFunc2' }, 1, 2);
10 bindFunc2(3, 4);
11 bindFunc2(5, 6);
12
```

OUTPUT TERMINAL DEBUG CONSOLE PROBLEMS

C:\Program Files\nodejs\node.exe test.js

{ x: 'bindFunc1' } 1 2 3 4

{ x: 'bindFunc2' } 1 2 3 4

{ x: 'bindFunc2' } 1 2 5 6

bind 메소드



● name 속성

- bind 메소드를 통해 새로 만든 함수는 name 속성에 동사 bind의 수동태인 'bound'라는 접두어가 붙음

```
1
2  var func = function (a, b, c, d) {
3      console.log(this, a, b, c, d);
4  };
5
6  var bindFunc = func.bind({ x: 'bindFunc' }, 4, 5);
7  console.log(func.name);
8  console.log(bindFunc.name);
9
```

OUTPUT TERMINAL DEBUG CONSOLE PROBLEMS

```
C:\Program Files\nodejs\node.exe test.js
func
bound func
```

상위 컨텍스트의 this를 내부 함수나 콜백 함수에 전달하기



```
1
2  var obj = {
3
4      outer: function () {
5
6          console.log(this);
7
8          var innerFunc = function() {
9              console.log(this);
10         };
11
12         innerFunc.call(this);
13     }
14 };
15
16 obj.outer();
17
```

OUTPUT TERMINAL DEBUG CONSOLE PROBLEMS

```
C:\Program Files\nodejs\node.exe test.js
{ outer: [Function: outer] }
{ outer: [Function: outer] }
```

```
1
2  var obj = {
3
4      outer: function() {
5
6          console.log(this);
7
8          var innerFunc = function() {
9              console.log(this);
10         }.bind(this);
11
12         innerFunc();
13     }
14 };
15
16 obj.outer();
17
```

OUTPUT TERMINAL DEBUG CONSOLE PROBLEMS

```
C:\Program Files\nodejs\node.exe test.js
{ outer: [Function: outer] }
{ outer: [Function: outer] }
```


상위 컨텍스트의 this를 내부 함수나 콜백 함수에 전달하기



```
1
2 var obj = {
3
4   logThis: function () {
5     | console.dir(this, { depth: 0 });
6   },
7
8   logThisLater1: function () {
9     | setTimeout(this.logThis, 500);
10  },
11
12  logThisLater2: function () {
13    | setTimeout(this.logThis.bind(this), 1000);
14  }
15 };
16
17 obj.logThisLater1();
18 obj.logThisLater2();
19
```

```
Timeout {
  _idleTimeout: 500,
  _idlePrev: null,
  _idleNext: null,
  _idleStart: 22,
  _onTimeout: [Function: logThis],
  _timerArgs: undefined,
  _repeat: null,
  _destroyed: false,
  [Symbol(refed)]: true,
  [Symbol(asyncId)]: 2,
  [Symbol(triggerId)]: 1
}
{
  logThis: [Function: logThis],
  logThisLater1: [Function: logThisLater1],
  logThisLater2: [Function: logThisLater2]
}
```

화살표 함수의 예외사항



- ES6에 새롭게 도입된 화살표 함수는 실행 컨텍스트 생성 시 `this` 를 바인딩하는 과정이 제외되었음
- 함수 내부에는 `this`가 아예 없으며, 접근하고자 하면 스코프 체인의 가장 가까운 `this`에 접근하게 됨

```
1
2  var obj = {
3
4      outer: function () {
5
6          console.log(this);
7
8          var innerFunc = () => {
9              console.log(this);
10         };
11
12         innerFunc();
13     }
14 };
15
16 obj.outer();
17
```

```
{ outer: [Function: outer] }
{ outer: [Function: outer] }
```

별도의 인자로 this를 받는 경우 (콜백 함수 내에서의 this)



```
1
2   var report = {
3       sum: 0,
4       count: 0,
5       add: function () {
6           var args = Array.prototype.slice.call(arguments);
7           args.forEach(function (entry) {
8               this.sum += entry;
9               ++this.count;
10          }, this);
11      },
12      average: function () {
13          return this.sum / this.count;
14      }
15  };
16
17  report.add(60, 85, 95);
18  console.log(report.sum, report.count, report.average());
19
```

OUTPUT TERMINAL DEBUG CONSOLE PROBLEMS

C:\Program Files\nodejs\node.exe test.js

240 3 80

별도의 인자로 this를 받는 경우 (콜백 함수 내에서의 this)



```
Array.prototype.forEach(callback[, thisArg])  
Array.prototype.map(callback[, thisArg])  
Array.prototype.filter(callback[, thisArg])  
Array.prototype.some(callback[, thisArg])  
Array.prototype.every(callback[, thisArg])  
Array.prototype.find(callback[, thisArg])  
Array.prototype.findIndex(callback[, thisArg])  
Array.prototype.flatMap(callback[, thisArg])  
Array.prototype.from(arrayLike[, callback[, thisArg]])  
Set.prototype.forEach(callback[, thisArg])  
Map.prototype.forEach(callback[, thisArg])
```

정리



● 다음 규칙은 명시적 `this` 바인딩이 없는 한 항상 성립함

- 전역 공간에서의 `this`는 전역 객체를 참조
 - (브라우저 환경: `window`, Node.js: `global`)
- 어떤 함수를 메소드로서 호출한 경우 `this`는 호출 주체를 참조
- 어떤 함수를 함수로서 호출한 경우 `this`는 전역 객체를 참조
 - 메소드의 내부 함수에서도 같음
- 콜백 함수 내부에서의 `this`는 해당 콜백 함수의 제어권을 넘겨 받은 함수가 정의한 바에 따르며, 정의하지 않은 경우 전역 객체를 참조
- 생성자 함수에서의 `this`는 생성될 인스턴스를 참조

정리



- 명시적 this 바인딩, 위 규칙에 부합하지 않는 경우에는 다음 내용을 통해 this를 예측할 수 있음
 - call, apply 메소드는 this를 명시적으로 지정하면서 함수 또는 메소드를 호출
 - bind 메소드는 this 및 함수에 넘길 인수를 일부 지정해서 새로운 함수를 만듦
 - 요소를 순회하면서 콜백 함수를 반복 호출하는 내용의 일부 메소드는 별도의 인자로 this를 받기도 함



Thank You!

